Advanced Problem Solving Patterns

Sliding Window • Two Pointers • Bit Manipulation • Segment Trees

Java 8+

Session Structure

1. Sliding Window Pattern
2. Two Pointer Pattern
3. Bit Manipulation Fundamentals
4. Segment Trees
5. Real-world applications
6. Complexity deep dive

Sliding Window

Used for:

- Subarrays
- Continuous sequences
- Optimize $O(n^2) \rightarrow O(n)$

Fixed Size Sliding Window

Example:

Max sum of subarray of size k.

Time: O(n)

```
static int maxSumK(int[] arr, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++)
        sum += arr[i];

    int max = sum;

    for (int i = k; i < arr.length; i++) {
        sum += arr[i] - arr[i - k];
        max = Math.max(max, sum);
    }

    return max;
}
```

Variable Size Sliding Window

Example:

Longest substring without repeating characters.

Time: O(n)

```
static int lengthOfLongestSubstring(String s) {
    java.util.Set<Character> set =
        new java.util.HashSet<>();

    int left = 0, max = 0;

    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left++));
        }
```

```
            set.add(s.charAt(right));
            max = Math.max(max, right - left + 1);
        }

        return max;
    }
```

Two Pointer Technique

Used for:

- Sorted arrays
- Opposite direction traversal
- Partitioning problems

Two Sum (Sorted)

Time: O(n)

```
static boolean twoSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left < right) {
        int sum = arr[left] + arr[right];

        if (sum == target) return true;
        if (sum < target) left++;
        else right--;
    }

    return false;
}
```

Remove Duplicates In-place

Time: O(n)

```
static int removeDuplicates(int[] arr) {
    int i = 0;

    for (int j = 1; j < arr.length; j++) {
        if (arr[j] != arr[i]) {
            arr[++i] = arr[j];
        }
    }

    return i + 1;
}
```

Bit Manipulation Basics

Binary representation of numbers.

Example:

5 → 101

Bitwise Operators

Operator	Meaning
&	AND
|	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Check Even or Odd

```
static boolean isEven(int n) {
    return (n & 1) == 0;
}
```

Time: O(1)

Count Set Bits

```java
static int countBits(int n) {
    int count = 0;

    while (n > 0) {
        n &= (n - 1);
        count++;
    }

    return count;
}
```

Time: O(number of set bits)

Single Number (XOR Trick)

Find element appearing once.

Time: O(n)

```
static int singleNumber(int[] arr) {
    int result = 0;

    for (int num : arr)
        result ^= num;

    return result;
}
```


Subset Generation using Bitmask

Time: O(2^n)

```
static void subsets(int[] nums) {
    int n = nums.length;

    for (int mask = 0; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0)
                System.out.print(nums[i] + " ");
        }
        System.out.println();
    }
}
```

Segment Tree

Used for:

- Range queries
- Efficient updates

Time:

Build → O(n)

Query → O(log n)

Update → O(log n)

Segment Tree Structure

Array-based representation.

Size ≈ 4n.

Build Segment Tree

```
static void build(int[] arr, int[] tree,
        int node, int start, int end) {

    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(arr, tree, 2 * node, start, mid);
        build(arr, tree, 2 * node + 1, mid + 1, end);

        tree[node] =
            tree[2 * node] + tree[2 * node + 1];
    }
}
```


Query Segment Tree

```
static int query(int[] tree,
        int node, int start, int end,
        int l, int r) {

    if (r < start || l > end)
        return 0;

    if (l <= start && end <= r)
        return tree[node];

    int mid = (start + end) / 2;

    return query(tree, 2 * node, start, mid, l, r)
         + query(tree, 2 * node + 1,
                 mid + 1, end, l, r);
}
```


Update Segment Tree

```
static void update(int[] tree,
        int node, int start, int end,
        int idx, int val) {

    if (start == end) {
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;

        if (idx <= mid)
            update(tree, 2 * node, start, mid,
                   idx, val);
        else
            update(tree, 2 * node + 1,
                   mid + 1, end, idx, val);

        tree[node] =
            tree[2 * node] +
            tree[2 * node + 1];
    }
}
```


Real World Use Cases

Sliding Window:

Network monitoring, streaming analytics.

Two Pointers:

Financial data analysis.

Bit Manipulation:

Cryptography, compression.

Segment Trees:

Stock market range queries,

Database analytics.

Complexity Summary

Pattern	Time
Sliding Window	O(n)
Two Pointers	O(n)
Bit Operations	O(1)
Segment Tree Query	O(log n)

Common Mistakes

- Forgetting window shrink logic
- Using sliding window on non-contiguous problems
- Misusing bit shifts
- Incorrect tree indexing
- Not handling boundary conditions

Interview Strategy

Ask:

1. Is subarray contiguous? → Sliding window
2. Sorted array? → Two pointers
3. Need fast bit tricks? → Bit manipulation
4. Frequent range queries? → Segment tree

Summary

These patterns provide:

- Performance optimization
- Advanced problem solving
- Scalable system design thinking

Master:

- Sliding Window
- Two Pointers
- Bit tricks
- Segment Tree construction